

Async Code – Professional Checklist

مع JavaScript / Node.js، asynchronous قبل كتابة أي كود **كل القواعد الأساسية للمحترفين** هذا الملف يجمع أمثلة عملية لكل نقطة.

1 تحديد طبيعة العملية

- **حسابية سريعة** أو عملية (DB, API, file) **I/O** حدد إذا كانت العملية []
- `async/await` أو `Promise` قرر ما إذا كنت ستستخدم []

```
// I/O → async/await
const users = await db.query("SELECT * FROM users");

// حساب سريع → synchronous
const total = items.reduce((a,b)=>a+b,0);
```

2 التعامل مع النتائج

- `then/await` اربط كل منطق يعتمد على النتيجة داخل []
- تجنب استخدام متغير خارجي قبل اكتمال العملية []

```
// ❌ race condition
let cache;
db.query("SELECT * FROM users").then(res => cache = res.rows);
if (cache) console.log("Using cache");

// ✅ ربط مباشر بالنتيجة
const users = await db.query("SELECT * FROM users");
console.log("Using cache");
```

3 إدارة الأخطاء

- `try/catch` مع `await` استخدم []
- `Promise` مع `.catch()` استخدم []

```
try {
  const users = await db.query("SELECT * FROM users");
} catch (err) {
```

```
console.error("DB query failed", err);
}
```

4 فهم Event Loop

- [] Microtasks (Promise.then, queueMicrotask, async/await) تُنفَّذ قبل Macrotasks (setTimeout, setInterval, I/O callbacks)
- [] Macrotasks و Microtasks ضع في ذهنك ترتيب التنفيذ عند دمج []
- [] لا تعتمد على ترتيب الأكواد ظاهريًا []

5 Race Conditions تجنب

- [] بالتسلسل إذا كانت النتائج متعددة await أو Promise.all استخدم []

```
// ✅ تنفيذ متوازي للنتائج
const [users, orders] = await Promise.all([
  db.query("SELECT * FROM users"),
  db.query("SELECT * FROM orders")
]);
```

6 تأكيد العودة أو الاستجابة

- [] server code يجب أن ترجع قيمة أو ترسل استجابة في async كل دالة []
- [] res.json() أو بدون return بدون async function لا تترك []

```
app.get('/users', async (req, res) => {
  const users = await db.query("SELECT * FROM users");
  res.json(users.rows);
});
```

7 وضوح وسهولة القراءة

- [] للكود المتسلسل async/await استعمل []
- [] للأحداث المنفصلة then/catch استعمل []
- [] معقد Microtask/Macrotask ضع التعليقات عند تسلسل []

8 المتراكمة callbacks تجنب تسرب الذاكرة أو

- غير منتظر تنفيذها Promises لا تترك []
- عند الانتهاء setInterval أو setTimeout تأكد من تنظيف []

```
const timer = setInterval(() => console.log("tick"), 1000);  
clearInterval(timer); // عند الانتهاء
```

♦ الخلاصة الذهبية

Always link your logic to the result, not the timing. بالبيانات التي async اربط كل خطوة. تنتجها، لا بالوقت المتوقع لوصولها

📄 JavaScript أو Node.js في asynchronous استخدم هذا الملف كمرجع دائم عند كتابة أي كود